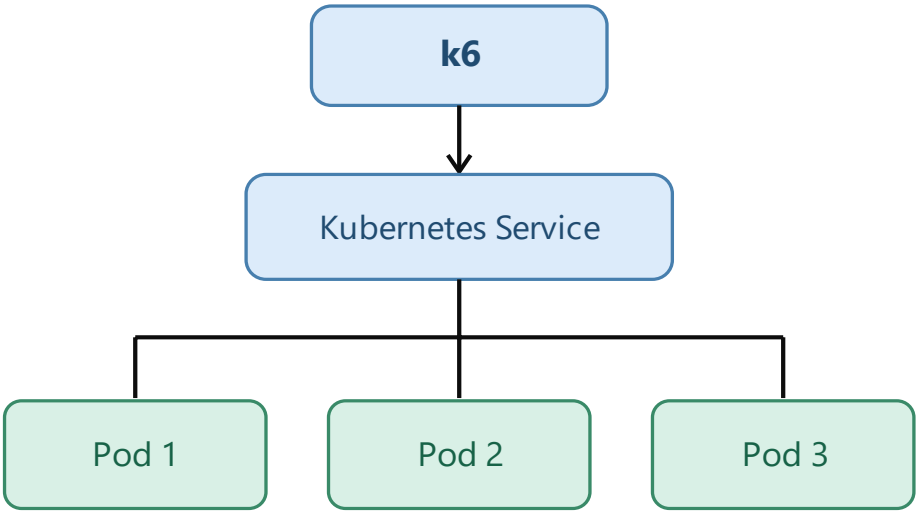
Лабораторная работа №4
Горизонтальное масштабирование веб-приложения в Kubernetes и оценка его эффективности

Цель работы:

– изучить влияние горизонтального масштабирования на производительность веб-приложения;
– научиться изменять количество реплик приложения в Kubernetes;
– исследовать зависимость RPS и времени ответа от количества реплик;
– использовать k6 для сравнительного нагрузочного тестирования;
– анализировать результаты с помощью Prometheus и Grafana;
– оценивать эффективность масштабирования и выявлять его ограничения.

Горизонтальное масштабирование – способ увеличения вычислительных возможностей системы посредством добавления экземпляров приложения, совместно обрабатывающих поступающие запросы.

В Kubernetes количество экземпляров приложения обычно задаётся параметром replicas ресурса Deployment.



Несколько экземпляров обрабатывают нагрузку

Увеличение количества Pod не гарантирует пропорционального роста производительности. Эффективность масштабирования зависит от:

– возможности независимой обработки запросов;
– распределения нагрузки между экземплярами;
– доступных CPU и памяти;
– наличия общих ограничивающих ресурсов;
– характера выполняемых приложением операций.

Предположим, один экземпляр приложения способен обрабатывать 100 запросов в секунду. В идеальных условиях четыре независимых экземпляра могли бы обрабатывать приблизительно 400 запросов в секунду. Однако на практике производительность может увеличиться значительно меньше или вообще остаться прежней.

Основные причины:

– ограниченное количество CPU и памяти в кластере;
– общий узкий ресурс, например база данных;

– блокировки и синхронизация при обработке запросов;
– неравномерное распределение нагрузки между Pod;
– дополнительные сетевые и вычислительные расходы;
– недостаточная интенсивность входящей нагрузки;
– ограниченная производительность самого генератора нагрузки.

Следовательно, эффективность горизонтального масштабирования необходимо оценивать экспериментально, а не только по количеству запущенных реплик.

Метрики эффективности масштабирования

Для оценки производительности используются метрики, изученные в предыдущих лабораторных работах.

Метрика	**Назначение**
RPS	Фактическое количество HTTP-запросов в секунду
p95 Latency	Время ответа, не превышаемое 95% запросов
P99 Latency	Время ответа, не превышаемое 99% запросов
Error Rate	Доля неуспешных запросов
CPU Usage	Использование процессорных ресурсов
Memory Usage	Использование оперативной памяти
Replicas	Количество экземпляров приложения

При увеличении числа реплик необходимо оценивать не только рост RPS, но и изменение задержек, ошибок и использования ресурсов.

Ускорение и эффективность масштабирования

Коэффициент ускорения характеризует увеличение максимальной устойчивой пропускной способности после изменения количества реплик:

$$S(N) = RPSmax(N) / RPSmax(1)$$

где
RPSmax(N) – максимальная устойчивая пропускная способность при N репликах; RPSmax(1) – максимальная устойчивая пропускная способность одной реплики.

Эффективность масштабирования определяется выражением:

$$E(N) = S(N) / N \times 100\%$$

Например, если одна реплика обрабатывает 100 RPS, а четыре – 320 RPS:

$$S(4) = 320 / 100 = 3,2;$$
$$E(4) = (3,2 / 4) \times 100\% = 80\%.$$

Таким образом, после увеличения числа реплик в четыре раза производительность выросла в 3,2 раза, а эффективность масштабирования составила 80%.

Примечание! В расчётах необходимо использовать максимальную устойчивую пропускную способность, полученную при одинаковых критериях допустимой задержки и ошибок. Сравнение RPS при заведомо недостаточной нагрузке не позволяет корректно оценить эффективность масштабирования.

Подготовка окружения

В работе используется окружение предыдущих лабораторных работ:

- Kubernetes – развёртывание приложения;
- k6 – генерация нагрузки;
- Prometheus – сбор метрик;
- Grafana – визуализация результатов.

Шаг 1. Проверка компонентов

Выполнить:

```
kubectl get nodes
kubectl get deployments
kubectl get pods
kubectl get services
kubectl get pods -n monitoring
```

Убедиться, что приложение и компоненты мониторинга работают.

В дальнейших примерах используются *условные имена:*

- `Deployment: web-app;`
- `Service: web-app-service;`
- `Namespace: default.`

Шаг 2. Подготовка приложения

Для исследования рекомендуется использовать Flask-приложение с endpoint, *выполняющим вычисления*, а не *простую отдачу статической страницы.*

Например, добавить в существующее приложение:

```
import hashlib
import os
import time
from flask import jsonify, request
from prometheus_client import Counter, Histogram

WORK_REQUESTS = Counter(
    "work_requests_total",
    "Number of completed work requests",
    ["pod"]
)
WORK_DURATION = Histogram(
    "work_duration_seconds",
    "Duration of CPU-bound work",
```

```
    ["pod"]
)
POD_NAME = os.environ.get("HOSTNAME", "unknown")

@app.route("/work")
def work():
    iterations = request.args.get(
        "iterations", default=20000, type=int
    )
    if iterations is None or not 1000 <= iterations <= 100000:
        return jsonify(error="Invalid iterations"), 400
    start = time.perf_counter()
    data = b"highload"
    for _ in range(iterations):
        data = hashlib.sha256(data).digest()
    duration = time.perf_counter() - start
    WORK_REQUESTS.labels(pod=POD_NAME).inc()
    WORK_DURATION.labels(pod=POD_NAME).observe(duration)
    return jsonify(
        pod=POD_NAME,
        duration=duration,
        result=data.hex()
    )
```

Если в приложении уже настроены `/metrics` и сбор метрик Prometheus, их необходимо сохранить.

Endpoint `/compute` создаёт вычислительную нагрузку на CPU, что позволяет исследовать изменение производительности при увеличении количества экземпляров приложения.

Количество итераций при *необходимости подобрать* так, чтобы одна реплика демонстрировала измеримую нагрузку без чрезмерно долгого выполнения запроса. Во всех экспериментах оно должно оставаться одинаковым.

Примечание! локальный Kubernetes может размещать все Pod на одном узле. В таком случае реплики разделяют общие ресурсы, поэтому *масштабирование может не дать существенного прироста производительности.*

Шаг 3. Установка одной реплики

```
kubectl scale deployment web-app --replicas=1
kubectl rollout status deployment/web-app
kubectl get pods
```

Убедиться, что приложение работает с одной репликой.

Шаг 4. Проверка доступности

Для проверки приложения (например(:

```
kubectl port-forward service/web-app-service 8080:80
```

Открыть:

`http://localhost:8080/compute`
Для сравнительных нагрузочных испытаний лучше использовать стабильную точку входа через NodePort или Ingress. `port-forward` может сам стать ограничивающим фактором.

Подготовка нагрузочного теста

Создать файл *test-scaling.js:*

```
import http from 'k6/http';
import { check } from 'k6';

const URL = __ENV.TARGET_URL ||
  'http://localhost:8080/compute';

export const options = {
  scenarios: {
    scaling: {
      executor: 'constant-arrival-rate',
      rate: Number(__ENV.RATE || 10),
      timeUnit: '1s',
      duration: '60s',
      preAllocatedVUs: 50,
      maxVUs: 200,
    },
  },

  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<500'],
  },
};

export default function () {
  const response = http.get(URL);

  check(response, {
    'status is 200': (r) => r.status === 200,
  });
}
```

В сценарии используется `constant-arrival-rate`, позволяющий задавать целевую интенсивность запуска итераций.

Поскольку одна итерация содержит один HTTP-запрос, целевая интенсивность *приблизительно* соответствует целевому RPS, если генератор нагрузки успевает выполнять сценарий.

Запустить проверочный тест:

```
k6 run -e RATE=10 test-scaling.js
```

Убедиться, что запросы выполняются и приложение возвращает HTTP 200.

Обратить внимание на `dropped_iterations`: если значение больше нуля, k6 не смог запустить часть запланированных итераций, что необходимо учитывать при анализе.

Задания для самостоятельного выполнения

Задание 1. Исследование производительности одной реплики

1. Установить одну реплику приложения.
2. Последовательно выполнить нагрузочные тесты с интенсивностью 10, 25, 50 и 100 запросов/с.
3. Для каждого теста зафиксировать фактический RPS, p95, Error Rate и CPU.
4. Определить максимальную устойчивую пропускную способность одной реплики.

Команда запуска:

```
k6 run -e RATE=25 test-scaling.js
```

При необходимости изменить диапазон интенсивности, если приложение достигает насыщения раньше или позже.

Результат: заполнить таблицу.

Целевой RPS	**Фактический RPS**	**p95, мс**	**Errors, %**	**CPU, cores**
10				
25				
50				
100				

Максимальная устойчивая пропускная способность – наибольшая измеренная интенсивность, при которой выполняются критерии p95 < 500 мс, Error Rate < 1%, отсутствуют существенные `dropped_iterations` и система не демонстрирует продолжающегося накопления задержек.

Задание 2. Исследование горизонтального масштабирования

Изменить количество реплик приложения:

```
kubectl scale deployment web-app --replicas=2
kubectl rollout status deployment/web-app
```

Проверить:

```
kubectl get pods -l app=web-app
```

Повторить нагрузочные тесты с теми же интенсивностями.

Затем увеличить количество реплик до трёх. Повторить измерения.

Для каждой конфигурации определить максимальную устойчивую пропускную способность.

Результат: заполнить сводную таблицу.

Реплики	**Максимальный устойчивый RPS**	**p95, мс**	**Errors, %**
1			
2			
3			

Выполнять испытания при одинаковых условиях и использовать один и тот же сценарий.

Задание 3. Анализ результатов в Grafana

Использовать dashboard, созданный в ЛР3.

Во время испытаний наблюдать:
- RPS;
- p95 Latency;
- Error Rate;
- CPU Usage;
- Memory Usage.

Дополнительно можно создать панель количества доступных реплик:

```
kube_deployment_status_replicas_available{
  deployment="web-app"
}
```

Для просмотра CPU отдельных Pod:

```
sum(
  rate(container_cpu_usage_seconds_total{
    container="web-app"
  }[1m])
) by (pod)
```

Убедиться, что фильтры соответствуют фактическим labels приложения.
Результат: представить графики и объяснить, как изменяются задержки и использование CPU при увеличении количества реплик.

Задание 4. Оценка эффективности масштабирования

Используя результаты задания 2, рассчитать:
- коэффициент ускорения;
- эффективность масштабирования.

Заполнить таблицу:

Реплики	**Устойчивый RPS**	**Ускорение S(N)**	**Эффективность E(N), %**
1		1,00	100
2			
3			

Ответить в выводах:
1. Увеличивается ли производительность пропорционально числу реплик?
2. Какие ограничения обнаружены?
3. Целесообразно ли дальнейшее увеличение количества реплик?

Если рост производительности отсутствует, определить возможную причину на основании метрик, не приписывая её автоматически приложению или Kubernetes.

Контрольные вопросы

1. Чем горизонтальное масштабирование отличается от вертикального?
2. Почему увеличение количества Pod не гарантирует пропорционального роста RPS?
3. Почему для сравнения конфигураций необходимо использовать одинаковые условия нагрузочного тестирования?
4. Что характеризуют коэффициент ускорения и эффективность масштабирования?
5. Как определить, что приложение достигло предела устойчивой производительности?
6. Почему масштабирование может оказаться неэффективным даже при корректной работе Kubernetes?